Introduction

A `ggplot2` theme controls everything in a plot that is not the data itself — panel background, axis ticks, gridlines, font sizes, legend placement, plot margins. Two layers of control coexist: a **complete theme** (`theme_minimal()`, `theme_bw()`, `theme_classic()`, etc.) sets every non-data element at once and is usually the right starting point, while individual `theme()` calls override specific elements for publication polish. Mastering both layers is what turns a default plot into one that looks at home in a journal article or a polished report.

Prerequisites

A working knowledge of `ggplot2`'s grammar of graphics, layered geoms, and aesthetic mappings.

Theory

Complete themes are functions that return a fully populated `theme` object. Built-in choices include `theme_minimal()` (white background, light gridlines, the de facto modern default), `theme_bw()` (white panel with thin black border), `theme_classic()` (axes only, no gridlines, mimicking print journals), and `theme_void()` (no axes or gridlines, useful for maps and decorative composites). The `ggthemes` package adds journal-flavoured options (`theme_tufte()`, `theme_economist()`, `theme_few()`).

Element overrides come from `theme()`, which accepts named arguments for every theme element. Each element is built from one of four constructors: `element_text()` for text (size, family, face, colour, hjust, vjust, margin), `element_line()` for lines, `element_rect()` for rectangles, and `element_blank()` to hide an element entirely. Common overrides include `legend.position`, `axis.title`, `panel.grid.minor`, and `plot.title`.

`theme_set()` applies a theme globally to every subsequent plot in the session, useful for enforcing house style across a multi-figure report.

Assumptions

No statistical assumptions; the only practical constraint is that font sizes, line widths, and margins must remain legible at the final print size, which is best verified by saving the plot at the target dimensions before judging it on screen.

R Implementation

```
library(ggplot2); library(ggthemes)

p <- ggplot(mtcars, aes(wt, mpg, colour = factor(cyl))) +
  geom_point(size = 3)

# Complete themes
p + theme_minimal()
p + theme_bw()
p + theme_classic()
p + theme_tufte()

# Custom theme
p +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 16, face = "bold"),
    axis.title = element_text(size = 12),
```

```
    legend.position = "bottom",
    panel.grid.minor = element_blank()
  ) +
  labs(title = "MPG vs weight", x = "Weight (1000 lbs)", y = "MPG",
       colour = "Cylinders")

# Set a global theme
theme_set(theme_minimal(base_size = 12))
```

Output & Results

Side-by-side comparisons of the same scatterplot under four complete themes plus a fully customised version. The custom variant combines `theme_minimal()` as the base with overrides for title weight, axis title size, legend position, and gridline visibility — a typical recipe for a publication-ready figure.

Interpretation

A reporting sentence (figure caption): "All figures use `theme_minimal(base_size = 12)` with custom title and axis label sizes; legend position was moved to the bottom for horizontal compositions." Mention the theme only when it affects interpretation (e.g. removing gridlines deliberately to emphasise a contrast); otherwise it can be left to a methods section "figure preparation" line.

Practical Tips

- Set a global theme once at the top of your analysis script via `theme_set()`; the rest of the figures inherit it automatically and stay consistent.
- For biomedical publications, `theme_minimal(base_size = 12)` with `panel.grid.minor = element_blank()` is a sensible default; it reads well in print and on screen.
- Use `element_blank()` to hide an element entirely; setting `colour = NA` only makes it invisible while still occupying its space.
- Keep font choices simple: `theme(text = element_text(family = "Helvetica"))` (or the corresponding sans-serif default) avoids the rendering surprises that come with fancier typefaces.
- Always export with `ggsave()` at the final intended `width`, `height`, and `dpi` (300 for raster, 600+ for line art); judging a plot on a screen at the wrong dimensions hides legibility problems.
- For bespoke journal styles, build a wrapper function that returns a fully configured theme and use it instead of repeating the same `theme()` overrides in every script.

R Packages Used

`ggplot2` for the base theme system, `ggthemes` for journal- and Tufte-flavoured presets, `cowplot` for `theme_cowplot()` (a publication-oriented variant), and `extrafont` if you need access to non-default system fonts in PDF output.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- ggplot2 grammar and multi-panel layouts